

乱数を用いたプログラム

釧路工業高等専門学校情報工学科5年・情報工学実験II

氏名：クリスティアン・ハルジュノ

出席番号：21

学籍番号：234071

提出日：2025年7月21日

目次

1	初めに	1
2	背景	1
2.1	5つのシミュレーションを紹介	1
2.1.1	課題1: 7つのサイコロを振る	1
2.1.2	課題2: 半径7の円の面積	2
2.1.3	課題3: 異身長男女を一行に並べる	3
2.1.4	課題4: ジャンケンの割合	5
2.1.5	課題5: ビンゴのシミュレーション	7
2.2	シミュレーションの役割	9
2.3	モンテカルロシミュレーション	10
3	実験方法	10
3.1	ハードウェアと環境	10
3.2	ランダム関数	11
3.3	モジュロによるランダム関数のバイアス	13
3.4	実行数	15
3.5	移動平均による値の収束	17
3.6	ジャンケンシミュレーションに用いる機械学習	17
4	実験結果	18
4.1	課題1: 7つのサイコロが同じ目を出すシミュレーション	18
4.2	課題2: 半径7の円の面積	20

1 初めに

統計の世界において、ある事象が起こる「確率」の計算は $\frac{\text{条件を満たす場合の数}}{\text{全体の事象数}}$ という単純な式で表される。たとえば、コインを投げて表が出る確率は $\frac{1}{2}$ であり、これはコインに 2 面が存在するためである。しかし、実際にコインを 10 回投げしてみると、表が出る割合は偏って見えることがあり、 $1/3$ や $1/2$ といった結果になる場合もある。ある意味で、確率には運による偏りの要素が依然として存在すると言える。このため、シミュレーションを多数回繰り返すことによって、「運」や「ノイズ」の影響を最小限に抑え、より正確な結果を得ることが重要である。

このレポートでは、乱数を用いたプログラムの実装とその結果の分析を行う。具体的には、コイン投げやサイコロの目の出方など、様々なランダムな事象をシミュレートし、それらの結果を統計的に解析することで、理論的な確率と実際の結果との関係を明らかにすることを目的とする。

2 背景

2.1 5つのシミュレーションを紹介

2.1.1 課題 1: 7つのサイコロを振る

本シミュレーションでは、7 個の 6 面サイコロを投げ、すべてのサイコロが同じ目を出す確率を求める。また、サイコロの個数を変化させながらシミュレーションを実行し、その結果を比較する。



図 1: 7つの同じ面のサイコロ

理論的には、一つの均質なサイコロにおいて各面が出る確率は正確に $\frac{1}{6}$ である。しかし、サイコロの個数を増やすことで、確率の分布は大きく変化する。 n 個のサイコロが同じ目を出す確率を計算するために、式 1 を用いる。

$$P = \frac{s}{s^n} = \frac{1}{s^{n-1}} \quad (1)$$

- n 個のサイコロのすべての組み合わせは s^n 通り。

- 全てのサイコロが同じ出目になる場合の数は s 通り（全て 1、全て 2、…、全て s ）。
- よって、確率は次のようになります：

$$P = \frac{\text{一致する場合の数}}{\text{全体の組み合わせ数}} = \frac{s}{s^n} = \frac{1}{s^{n-1}}$$

本実験では 7 個の 6 面サイコロをシミュレーションするため、すべてのサイコロが同じ目を示す確率は式 2 に示す通り、 2.143×10^{-5} となる。

$$P = \frac{1}{6^{7-1}} = \frac{1}{6^6} = \frac{1}{46656} \approx 2.143 \times 10^{-5} \quad (2)$$

2.1.2 課題 2: 半径 7 の円の面積

本シミュレーションでは、2 次元平面上に半径 7 の円を描く。乱数により生成された位置 (x, y) のプロットを平面上に配置し、そのプロットが円の内部に存在する理論的確率を計算し、シミュレーション結果と比較する。

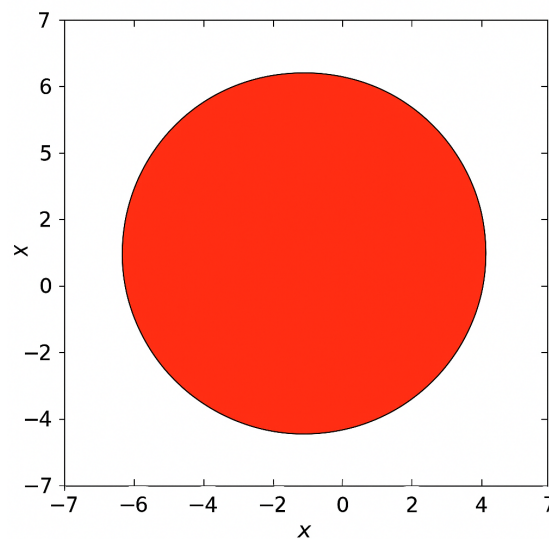


図 2: 半径 7 の円の面積

乱数によって生成されたプロットが円の内部に入る理論確率の計算は、式 3 に示すように、円の面積を全体の面積で割るだけである。本シミュレーションを簡略化するため、プロットの座標の上限および下限を -7 から 7 に制限し、面積を 14×14 に設定する。

$$P = \frac{\text{円の面積}}{\text{領域の面積}} = \frac{\pi r^2}{A} \quad (3)$$

この場合、全体の領域の一辺が 14 であることが分かっているため、面積は $A = 14 \times 14 = 196$ と計算できる。一方、円の半径を 7 に設定しているため、円の面積は $\pi \times r \times r = \pi \times 7 \times 7 = 49\pi$ と計算できる。これら二つの値を式 3 に代入することで、式 4 を得り、乱数によって生成されたプロットが円の内部に入る確率は約 0.785 となる。

$$P = \frac{49\pi}{196} = \frac{\pi}{4} \approx 0.785 \quad (4)$$

2.1.3 課題 3: 異身長男女を一行に並べる

本実験では、身長異なる 10 名（男性 5 名、女性 5 名）をランダムに一行に並べ、各女子の左側に、自身よりも背の高い男子が 1 人以上いる並びとなる確率をシミュレーションにより求める。

各人の身長の関係は以下のように定義されており、すべての男性および女性は明確な順序で区別されている：

$$M_1 > W_1 > M_2 > W_2 > M_3 > W_3 > M_4 > W_4 > M_5 > W_5 \quad (5)$$

この関係に従って、10 名を一行にランダムに並べる。その上で、すべての女性について、「自身より背の高い男性が左側に少なくとも 1 人存在する」かどうかを調べる。この条件をすべての女性が満たす並びが、全体の中でどの程度の確率で現れるかを、シミュレーションにより統計的に推定する。

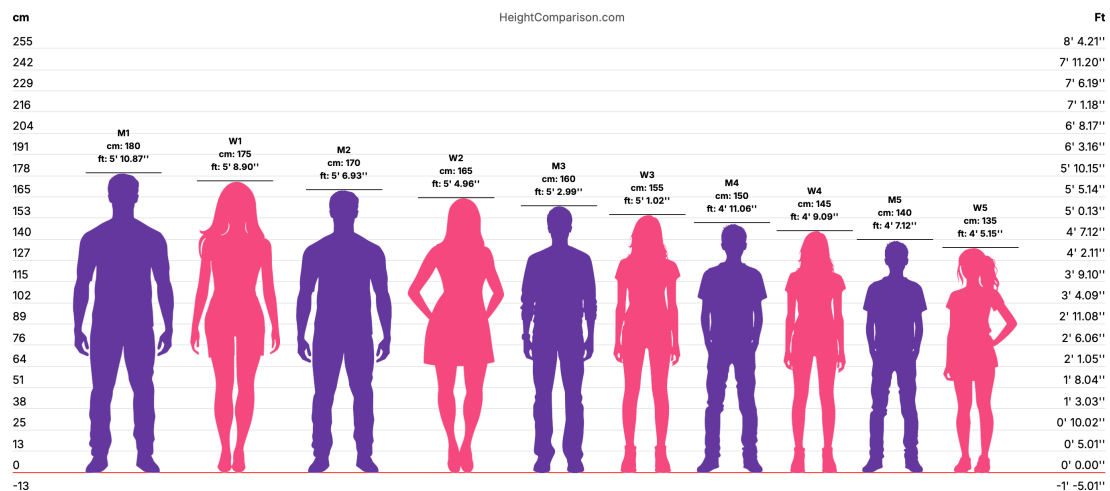


図 3: 男女 10 名の身長関係と識別

しかしながら、本問題の条件により、理論的な割合や期待値の計算は非常に複雑になる。先に述べた通り、列内の各女性に対して、少なくとも一人はその女性より身長の高い男性が左側に存在しなければならない。そのため、本シミュレーションでは 10 人の並びの全ての順列を総当たりで検証し、条件を満たすかどうかを確認する手法を用いる。

n 個の対象を一行に並べる場合の総順列数は、式 6 により計算できる。

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1 \quad (6)$$

今回のように、身長異なる 10 人（あるいは 10 個の対象）を一行に並べる場合、式 6 を用いることで、順列の総数は

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 3,628,800$$

通りであることが分かる。

次に、Python のコードを用いて、先に定義した条件に対して行のすべての順列を検証した。各順列をチェックするために使用した Python コードはコード 1 に示す。

コード 1: 順列を確認するコード

```
def is_valid(permutation):
    for i, person in enumerate(permutation):
        if person.startswith('W'):
            w_height = height_rank[person]
            left = permutation[:i]
            found_taller_man = any(p.startswith('M')
                                   and
                                   height_rank[p] < w_height for p in left)
            if not found_taller_man:
                return False
    return True
```

コードを各順列を確認する結果は表 1 に表されている。ブルートフォース法を用いて、条件を満たす順列の数が 893025 であることを確認した。有効な順列の数を全順列の数で割ることにより、理論的確率または期待値が 0.2461 となることが得られた。

表 1: 身長条件シミュレーションにおける順列結果

項目	値
総順列数	3,628,800
有効な順列数	893,025
確率（割合）	0.2461

2.1.4 課題 4: ジャンケンの割合

ジャンケンシミュレーションは、伝統的なジャンケンに得点制度を導入したシミュレーションである。提示された条件に従い、特定の手による勝敗によって得られる点数（本稿ではドル単位）が異なる。完全なルールは表 2 に示す。

表 2: じゃんけんの勝利手ごとの報酬（単位：ドル）

あなたの手 \ 相手の手	グー	パー	チョキ
グー (G)	0	-10	+10
パー (P)	+30	0	-30
チョキ (T)	-20	+20	0

ジャンケンに設定されたルールが複雑であるため、最も高い得点を得る各手（グー、チョキ、パー）の理論的な確率や期待される比率を正確に求めることはほぼ不可能である。しかし、式 7 を用いることで、その比率を近似的に求めることができる。

$$E = \sum_{i \in \{R, P, S\}} \sum_{j \in \{R, P, S\}} P_{\text{you}}(i) \cdot P_{\text{opp}}(j) \cdot \text{reward}(i \text{ vs } j) \quad \text{ここで } E \text{ は期待利益となる} \quad (7)$$

本初回のシミュレーションでは、相手の手は偏りなくランダムに選ばれる。そのため、各手の比率は正確に $\frac{1}{3}$ となる。先に表 2 で示した各手の勝敗に伴う得失を踏まえ、式 7 に代入することで、相手が偏りない場合に最も最適な勝率または戦略の比率を算出できる。

$$\begin{aligned}
E(r, p, s) &= r \cdot \left(\frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-30) + \frac{1}{3} \cdot (+10) \right) \\
&\quad + p \cdot \left(\frac{1}{3} \cdot (+30) + \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-20) \right) \\
&\quad + s \cdot \left(\frac{1}{3} \cdot (-10) + \frac{1}{3} \cdot (+20) + \frac{1}{3} \cdot 0 \right) \\
&= r \cdot \left(\frac{-30 + 10}{3} \right) + p \cdot \left(\frac{30 - 20}{3} \right) + s \cdot \left(\frac{-10 + 20}{3} \right) \\
&= r \cdot \left(\frac{-20}{3} \right) + p \cdot \left(\frac{10}{3} \right) + s \cdot \left(\frac{10}{3} \right) \\
&= \frac{1}{3} (-20r + 10p + 10s)
\end{aligned} \tag{8}$$

最大の利益 (E) を得るためには, $r + p + s = 1$ かつ $r, p, s \geq 0$ を満たす条件下で, $E(r, p, s)$ を最大化する必要がある. $r + p + s = 1$ であるため, 式 8 における s を $1 - r - p$ に置き換えることができ, これにより計算を式 9 の形に簡略化できる.

$$\begin{aligned}
E(r, p) &= \frac{1}{3} (-20r + 10p + 10(1 - r - p)) \\
&= \frac{1}{3} (-20r + 10p + 10 - 10r - 10p) \\
&= \frac{1}{3} (-30r + 10)
\end{aligned} \tag{9}$$

式 9 の内容を検討すると, r が増加するにつれて $E(r, p)$ の値は減少する. また, $E(r, p)$ を最大化する必要がある, $r \geq 0$ であることから, $E(r, p)$ は $r = 0$ のときに最大値をとる. 最後に, r を 0 に置き換えると, p と s の合計比率が 1 である条件のもとで, 式 10 に示すように期待利益は約 3.33 となる. この場合, p と s の任意の組み合わせ (例えば 0.2 と 0.8, 0.6 と 0.4 など) でも最大利益を得られることになる.

$$E(0, p) = \frac{1}{3} (0 + 10) = 3.33 \tag{10}$$

式 8 から式 10 にかけて行ったすべての計算は, 相手が特定の手を他の手よりも頻繁に出すといった偏りを持たないという前提に基づいている. この前提のもとでは, 各手 (グー, チョキ, パー) はそれぞれ約 0.33 の確率で出現すると仮定される.

しかし現実においては, 人間は学習を通じて自身の出す手の比率を調整し, 最大限の利得を得ようとする傾向がある. このような状況下では, 最適な手の出し方の比率を求めることがより複雑になる. というのも, 相手の勝利戦略自体が繰り返しの中で動的に変化するためである.

本実験では、より現実に近い状況を模擬するために、相手に機械学習を導入し、完全なランダムではない戦略を用いる。こうした環境下では、理論的に最適な勝利戦略を計算することがほぼ不可能であるため、モンテカルロ・シミュレーションの重要性がより一層高まる。

2.1.5 課題 5: ビンゴのシミュレーション

本シミュレーションでは、表 3 に示した条件に従い、ランダムに生成されたビンゴカードを作成する。

表 3: ビンゴカードの列と対応する数値範囲

列	数値範囲
1 列目 (B)	1 ~ 15
2 列目 (I)	16 ~ 30
3 列目 (N)	31 ~ 45
4 列目 (G)	46 ~ 60
5 列目 (O)	61 ~ 75

また、従来のビンゴのルールでは、フルビンゴを達成するために 5 列すべてを完成させる必要がある（各列につき 1 つ）。しかし本シミュレーションでは、1 列の完成をフルビンゴとみなす。完成した列は、ビンゴカード上の任意の位置における横、縦、または斜めのいずれかとなる。また、ビンゴカードの中央はフリースペースとされ、ゲーム開始時点で既にこのマスは開いているものとする。

ビンゴゲームにおいて、以下の 3 つの異なる条件がシミュレーションされた。

1. 7 回以内で当たる確率をシミュレーション

本シミュレーションは、7 回のコール以内当たる確率を求めることを目的とする。

2. 最短回数 (4 回) で当たりとなる確率をシミュレーション

本シミュレーションは、最短ターン数である 4 回以内（ビンゴカード中央が既に開いていることを考慮）に当たる確率を算出することを目的とする。

3. 当たりとなる最も遅いゲーム回数とそのような状態の発生確率をシミュレーション

本シミュレーションは、最も遅い（コール数が最大となる）タイミングで当たる確率を算出することを目的とする。ビンゴカードには、表 3 に示した通

り, 75 の候補番号のうち 24 個 (5×5 のマスから中央の 1 マスを除く) が記載されている. これは, ビンゴカード上の番号に当たらずに呼ばれることが可能な番号が $75 - 24 = 51$ 個存在することを意味する. ビンゴカードにない番号をすべて呼んだ後でも, 表 4 に示すように, ビンゴカード上で列が完成しないまま呼べる番号がさらに 19 個存在する.

表 4: 完全列または表を当たらないビンゴカード

B	I	N	G	O
O	X	X	X	X
X	X	O	X	X
X	X	X	O	X
X	O	X	X	X
X	X	X	X	O

つまり, 列や行が完成しないまま呼べる番号の総数は $51 + 19 = 70$ となる. 言い換えれば, 最悪の場合でも 71 回目のコールで必ずビンゴが達成されることになる.

これらのシミュレーションにおける正確な理論的確率や期待値を計算することは極めて複雑であり, 単一の数値を導出するための単一の方程式は存在しない.

しかし, 表 3 に示したビンゴカード作成のルールを考慮すると, 式 11 に示す組み合わせの公式を用いて, 可能な組み合わせ数を計算することができる.

$$nCr(n, r) = \frac{n!}{r!(n-r)!} \quad (11)$$

$$\text{B 列: } 15C5 = 3003$$

$$\text{I 列: } 15C5 = 3003$$

$$\text{N 列: } 15C4 = 1365 \quad (\text{中心の FREE スペースのため 4 つ選ぶ})$$

$$\text{G 列: } 15C5 = 3003$$

$$\text{O 列: } 15C5 = 3003$$

(12)

$$\text{合計のカード数: } (15C5)^4 \times 15C4 = 3003^4 \times 1365$$

$$\begin{aligned} \text{数値計算: } 3003^4 &\approx (3.003 \times 10^3)^4 = 3.003^4 \times 10^{12} \approx 81.2 \times 10^{12} \\ &= 8.12 \times 10^{13} \end{aligned}$$

$$8.12 \times 10^{13} \times 1365 \approx 1.1 \times 10^{17}$$

したがって、式 12 の計算により、ビンゴカードの総数は約 1.1×10^{17} 。この総数で条件を満たすカードを計算することは本手法は多大な計算資源を要する。それにより、各シミュレーション条件の確率を近似するためには、シミュレーションを用いる方が望ましい。

2.2 シミュレーションの役割

現実には、与えられた条件に対する理論的確率や期待値の導出は、ルールの変更やバイアスの導入によって困難になる場合がある。例えば、節 2.1.1 および 2.2.2 で示したように、6 面ダイスや単純な 2 次元円を用いた場合の理論的確率の算出は、シミュレーション自体で考慮すべき要素が少ないため容易である。しかし、ダイスに偏りがあり、特定の面が他の面よりも出やすいといった条件を導入した場合、理論的にはその偏りの度合いが正確に分かっていれば、バイアスを計算し理論値を求めることは可能である。とはいえ、実際には複数の予期しない要因が結果に影響を及ぼす可能性があり、このような解析ははるかに困難となる。[5]

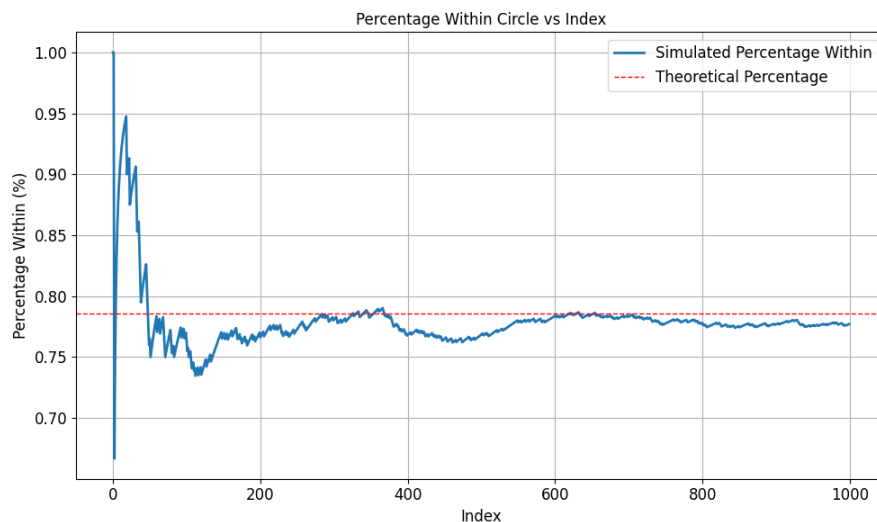


図 4: 出現率が収束する例

このような場合において、理論的確率を直接計算するのではなく、シミュレーションを用いることがはるかに有効な代替手段となる。シミュレーションでは、条件に従って同じ実験を何度も繰り返し実行し、有効な結果のみを記録すればよい。シミュレーションの回数を十分に重ねることで出現率は次第に安定し、図 4 に示すように、当該条件における出現確率のおおよその傾向を把握することが可能となる。

2.3 モンテカルロシミュレーション

モンテカルロシミュレーションとは、複雑な条件における確率を得るために繰り返しサンプリングを用いる計算アルゴリズムの一種である。節 2.2 で述べたように、条件によっては理論的な確率を正確に計算することが計算コストの面で非常に高く、多大な時間を要する場合がある。モンテカルロシミュレーションを用いることで、すべての順列を網羅的に列挙することなく、不確実な事象の発生確率を推定することが可能となる。たとえば、節 2.1.5 で定義された制約下におけるビンゴのシミュレーションでは、ビンゴカードの可能な順列数が 1.1×10^7 以上存在するため、全順列を評価するには数日から場合によっては数年を要する可能性がある。このような場合にモンテカルロシミュレーションを用いることで、当該事象の確率を現実的な計算時間内で推定することができる。[2]

3 実験方法

本節では、節 2.1 で述べた実験において使用される各種手法および関数について説明する。

3.1 ハードウェアと環境

ハードウェアとしては Apple シリコンのプロセッサに基づいて実験を行う。

機種名 Apple Macbook Pro (M3, 2023 年モデル)

プロセッサ Apple M3 チップ (8 コア：高性能コア 4 + 高効率コア 4)

GPU 10 コア統合型 GPU

メモリ 16GB ユニファイドメモリ

ストレージ 1TB SSD

OS macOS Sequoia 15.5

アーキテクチャ ARM64 (Apple シリコン)

また、作成した実験プログラムは C 言語で書き、GCC コンパイラでコンパイルされる。

Apple clang version 17.0.0 (clang-1700.0.13.5)

Target: arm64-apple-darwin24.5.0

Thread model: posix

コンパイルを効率化するため、GCC コンパイラを Make と同時に利用する。

GNU Make 3.81

Copyright (C) 2006 Free Software Foundation, Inc.

This is free software; see the source for copying conditions.

There is NO warranty; not even for MERCHANTABILITY or
FITNESS FOR A PARTICULAR PURPOSE.

This program built for i386-apple-darwin11.3.0

3.2 ランダム関数

C 言語において、乱数値を生成する最も一般的な方法の一つは、`stdlib.h` ライブラリが提供する `rand()` 関数である。`rand()` 関数は通常、乱数の種（シード）を設定するための `srand()` 関数と組み合わせて用いられる。一般に「乱数」とは、次にどの値が出るか予測できないことを意味するが、広く使われている `rand()` 関数の実装は厳密には真の乱数ではなく、疑似乱数である。先に述べたように、`rand()` 関数は `srand()` 関数と組み合わせて使用され、乱数生成の開始点となる種を設定する。この種は、疑似乱数生成の計算基準となる開始値である。この開始値は式 13 におく LCG (Linear Congruential Generator) であり、予め定められた定 x 数群を用いて次の疑似乱数を生成するために用いられる。

$$X_{n+1} = (aX_n + c) \bmod m \quad (13)$$

この式は `rand()` 関数内に実装されており、コード 2 に類似した形で表される。

コード 2: `rand()` 関数の简单版

```
static unsigned long next = 1;

int rand(void) {
    next = next * 1103515245 + 12345;
    return (unsigned int)(next / 65536) % 32768;
}

void srand(unsigned int seed) {
    next = seed;
}
```

一定の式を用いているため、乱数生成は開始種が分かっているならば、実際には生成される値を予測可能である。これは、`stdlib.h` ライブラリが提供する一般的な `rand()`

関数はエントロピーが低く、 m 回の出力後には必ず生成される値が繰り返されることを意味する.[1]

モンテカルロシミュレーションにおいて、この制約はシミュレーションの精度を著しく低下させる。なぜなら、可能なすべてのケースや順列を十分に一般化するために、シミュレーションを可能な限り多く繰り返す必要があるからである.[7]

この制約を回避するために、すべてのシミュレーションは、`stdlib.h` ライブラリが提供する `arc4random` の拡張である `arc4random_uniform` 関数を用いて実行する。`rand` 関数とは異なり、`arc4random` は LCG（線形合同法）を使用せず、より安全な CSPRNG（暗号学的に安全な疑似乱数生成器）を採用している。`rand` と `arc4random` はいずれも真の乱数ではなく疑似乱数であるが、`arc4random` はデバイスのハードウェアから得られるエントロピー値を乱数の種として利用する点異なる。一方で、一般的な `rand` 関数はエポック時刻を基に初期シードを設定しているため、実行時刻が正確にわかれば、次に生成される乱数を「予測」できる可能性がある.[8]

`arc4random` 関数を実行すると、デバイスは `/dev/urandom` から疑似乱数バイト列を読み取る。この `urandom` ファイルは、デバイスのカーネルが管理する内部エントロピープールから生成される。エントロピープールは、ネットワークタイミング、キーボードやマウスの入出力、割り込みタイミングなど、ハードウェアに基づくさまざまな値からシードされる。`/dev/urandom` から読み取った内容の一例をコード 3 に示す。

コード 3: `urandom` の内容のを読み取り結果例

```
head -c 16 /dev/urandom | xxd
00000000:
e690 1de3 6b5c 6c27 7684 fc82 6cfa a9a3
....k\l'v...l...
head -c 16 /dev/urandom | xxd
00000000:
c15c a4e2 c72e 8d0d 5494 4f21 874b d0b3
.\.....T.O!.K..
head -c 16 /dev/urandom | xxd
00000000:
3198 d4b2 7c21 5e2b 41a6 7a94 8012 f0ab
1...|!^+A.z.....
```

これらのランダムな値は、ChaCha20 アルゴリズムに入力される。ChaCha20 は、

Daniel J. Bernstein によって 2008 年に設計されたストリーム暗号であり、実用的な暗号解析に対して高い耐性を持つ。現代の暗号技術において広く用いられており、OpenSSH や TLS (Transport Layer Security) , 各種オペレーティングシステムなど、業界標準の技術においても利用されている.[6]

arc4random は、低レイヤのハードウェア層から生成されるエントロピー値を元にシードが設定されるという特性を持つ。このため、arc4random は通常の rand 関数と比較して、より「ランダム」に近い値を生成することが可能となる。

3.3 モジュロによるランダム関数のバイアス

ランダム値ジェネレータ内部には、値をユーザに返す前に行われる最終的な操作が存在する。この操作はモジュロチェックと呼ばれ、生成された値に対してあらかじめ設定された最大値で剰余演算を行うことで、値がその範囲を超えないように制限される。しかし、この手法はランダム値を特定の範囲内で生成したい場合に大きな問題を引き起こす.[3] 例えば、ランダム値ジェネレータが符号なし 8 ビット整数の最大値である 255 まで出力可能だとする。このとき、arc4random 関数に対して最大値を 120 と指定した場合、120 を超えるすべての数値は先頭から再び数えられるように変換される。これにより、予期せぬバイアスが発生し、特定の値が他の値よりも頻繁に出力される傾向を持つようになる。

arc4random と arc4random_uniform の違いを適切に評価するために、C 言語を用いて小規模なシミュレーションを実施した。arc4random は上下限の設定を直接サポートしないため、コード 4 に示すように再実装を行った。

コード 4: 再実装した arc4random 関数

```
int rand_in_range(int min, int max){
    return arc4random() % (max - min + 1) + min;
}
```

一方、arc4random_uniform は下限をサポートしないため、コード 5 に示すようにこちらも再実装を行った。

コード 5: 再実装した arc4random_uniform 関数

```
int rand_uniform_in_range(int min, int max){
    return arc4random_uniform(max - min + 1) + min;
}
```

両関数を下限 1, 上限 100 で 10 万回繰り返し実行した結果は, 図 5 に示すように類似している.

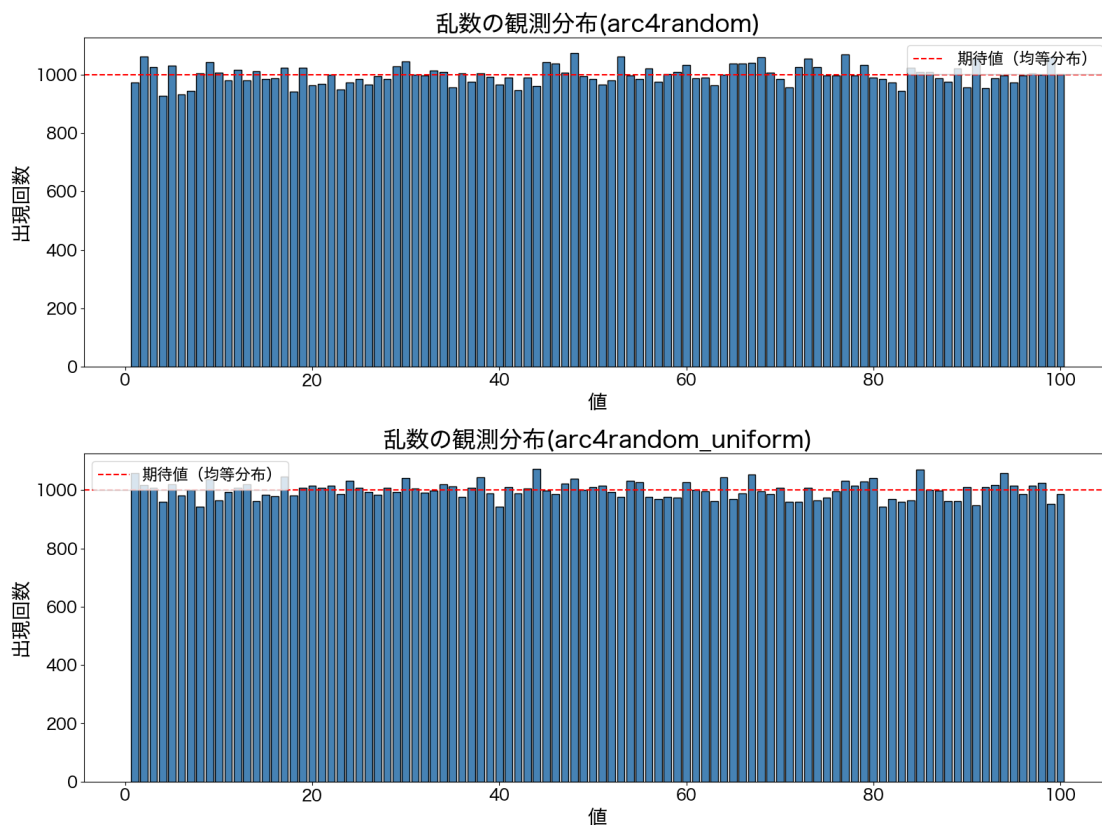


図 5: arc4random と arc4random_general によって生成された乱数の比較 (下限 1, 上限 100)

グラフをそのまま比較した場合, 2つの関数間に有意な差があるとは言い難い. 適切な比較を行うために, 両手法の p 値およびカイ二乗値を算出する.

表 5 の結果から, モンテカルロシミュレーションに必要な高いエントロピーを arc4random_uniform の結果において確認できる. エントロピーが高いほど, 生成される値はよりランダムであることを示す. 一方で, カイ二乗値が低いことは, この結果が一様分布に近いことを意味する.

両関数とも p 値に基づくバイアスは有意ではないものの, arc4random_uniform のエントロピースコアが高いことにより, シミュレーションデータに含まれる潜在的なノイズが除去される. 以上の理由から, 本実験ではすべてのシミュレーションに arc4random_uniform 関数を用いる.

表 5: カイ二乗検定の結果

メソッド	カイ二乗統計量	P 値
<i>arcRandom</i>	103.030000000	0.370758315
<i>arcRandom_uniform</i>	83.980000000	0.859645089

3.4 実行数

前節 2.3 で述べたように、モンテカルロシミュレーションでは理論値に近い結果を得るために大量の反復回数が必要となる。サイコロのような単純なシミュレーションでは 1000 回の反復で十分な場合もあるが、より複雑なルールセットや稀な事象を扱うシミュレーションでは、正確な結果を得るために 100 万回以上の反復が必要となることがある。適切な反復回数の設定は困難であり、多くの試行錯誤を要する。繰り返し回数の変化による影響は、図 6 と図 7 に示されている。この図では、2つのサイコロが毎回同じ目を出す確率のシミュレーションをそれぞれ 100 回および 1000 回繰り返した結果を比較している。

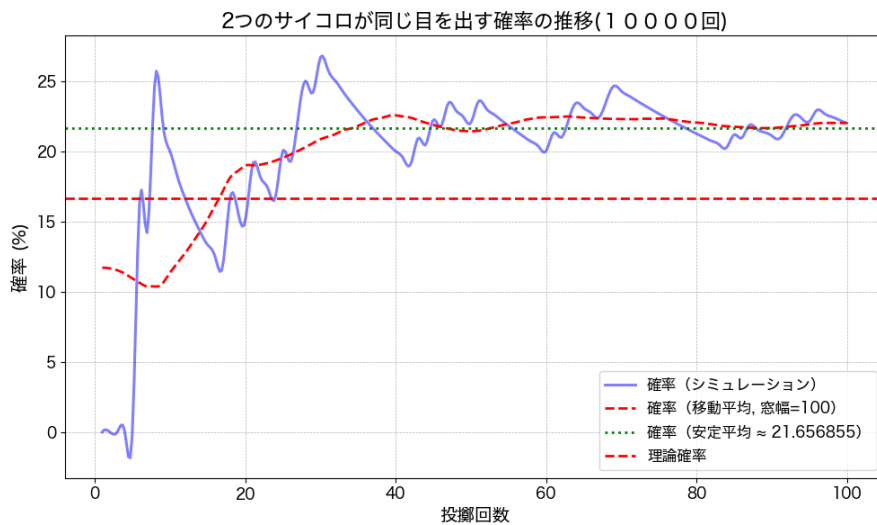


図 6: 二つのサイコロが同じ目を出す確率の比較 (100 回)

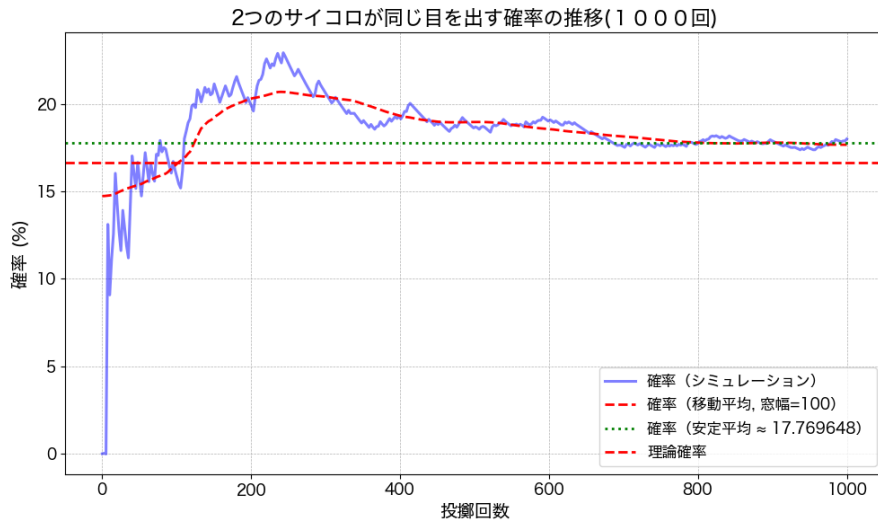


図 7: 二つのサイコロが同じ目を出す確率の比較 (1000 回)

2つのサイコロが1回の試行で同じ目を出す理論的な確率は, 単純に以下のように計算できる:

$$\frac{6}{6} \times \frac{1}{6} = \frac{1}{6} \approx 0.167 \approx 16.7\%$$

つまり, モンテカルロ・シミュレーションによっても同様の値が得られるはずである. 異なる繰り返し回数で実施した同一シミュレーションの比較結果は, 表 6 に示されている.

表 6: 投擲回数, 安定平均, および理論値 16.7%からの偏差率の比較

投擲回数	安定平均 (%)	理論値 16.7%からの偏差率 (%)
100	21.656855	$\frac{ 21.656855 - 16.7 }{16.7} \times 100 \approx 29.72$
1000	17.76948	$\frac{ 17.76948 - 16.7 }{16.7} \times 100 \approx 6.44$
10000	16.542044	$\frac{ 16.542044 - 16.7 }{16.7} \times 100 \approx 0.95$

この簡単な実験結果から, 繰り返し回数が多くなるほど理論的な期待値に近づくことが確認できる. しかしながら, 任意のシミュレーションに対して最適な繰り返し回数を一意に決定する標準的な手法は存在しない.

したがって, 本シミュレーションで扱うより複雑な実験においては, 繰り返し回数を手動で調整し, 移動平均による値の収束が最も安定するように設定を行う.

3.5 移動平均による値の収束

移動平均は、統計解析および時系列処理における基本的な手法である。この手法では、あらかじめ定めたウィンドウサイズに基づいて、時系列データ上をスライドさせながら局所的なデータ区間の平均値を計算する。

この処理により、短期的な変動が平滑化され、より長期的な傾向や周期性が浮き彫りになる。移動平均は、ノイズの多いデータから本質的なパターンを抽出する際に有効である。移動平均を用いることで、データに含まれる短期的なノイズを平滑化し、全体的な傾向とは無関係な変動を除去することができる。これにより、データ内の有意なパターンや潜在的なトレンドが明確に現れるようになる。[4]

移動平均の計算には、最も基本的な形態である単純移動平均 (Simple Moving Average, SMA) を用いる。SMA の計算式は式 14 に示す通りである。この式は、直近の w 個のデータを入力として受け取り、それらの平均値を一つの値として算出する。データを順次処理しながら SMA を繰り返し計算することで、シミュレーションの比率変動を平滑化する。

$$\text{SMA}_t = \frac{1}{w} \sum_{i=0}^{w-1} x_{t-i} \quad (14)$$

3.6 ジャンケンシミュレーションに用いる機械学習

前述のとおり (節 2.1.4 参照)、じゃんけんにおける現実に近い状況をシミュレートするため、機械学習を導入し、繰り返しごとに戦略を最適化する学習型の対戦相手を構築する。

本実験で使用するモデルは、以下の特性を持つように設計されている:

特徴ベクトル じゃんけんにおける次の手を予測するための 6 次元の入力ベクトル。以下の 2 種類の特徴量から構成される。

直前の手 (3 次元): 相手 (自分側) が最後に出した手を one-hot 表現したもの。グーであれば $[1, 0, 0]$ 、パーであれば $[0, 1, 0]$ 、チョキであれば $[0, 0, 1]$ と表現される。

セッション内の手の比率 (3 次元): セッション全体で相手 (自分側) が出した手の分布。各値は 0 から 1 の間で、3 つの値の合計は 1 になる。

モデル出力 次に出す手 (グー・パー・チョキ) それぞれに対応するスコアまたは確率を示す 3 次元の出力ベクトル。

グースコア: 次にグーを出す確率または信頼度。

パースコア: 次にパーを出す確率または信頼度.

チョキスコア: 次にチョキを出す確率または信頼度.

モデル構造 ソフトマックス関数を適用した単一の線形層で構成される. 隠れ層は存在せず, モデル構造は単純な多クラスロジスティック回帰に相当する. 可読性と学習の安定性を重視して設計されている.

4 実験結果

4.1 課題 1: 7つのサイコロが同じ目を出すシミュレーション

前節 2.1.1 で説明した通り, 7つのサイコロを投げた際に全て同じ目が出る理論的確率は約 2.143×10^{-5} である. 異なる投擲回数における出現率のサンプリング結果は表 7 に示されている.

表 7: 指定投擲回数における生データの出現率と理論確率に対する偏差率 (%)

投擲回数	出現率 (%)	偏差率 (%)
50,000	0.000000	-100.00
100,000	0.400000	86.74
500,000	0.240000	11.99
1,000,000	0.300000	39.96
5,000,000	0.222000	3.58
理論確率	0.214335	—

対象条件の出現率は, その時点での出現回数をそれまでの投擲回数で割ることにより計算される. 5つの異なる時点 (50,000 回から 5,000,000 回) でのサンプル率を取得した結果, 7つのサイコロが同じ目を出す出現率は, 前節で計算した理論値または期待値 2.143×10^{-5} に徐々に収束することが確認された.

繰り返し回数が増加するにつれて, 出現率はより理論値に近づいていく様子が, 図 8 に示されている.

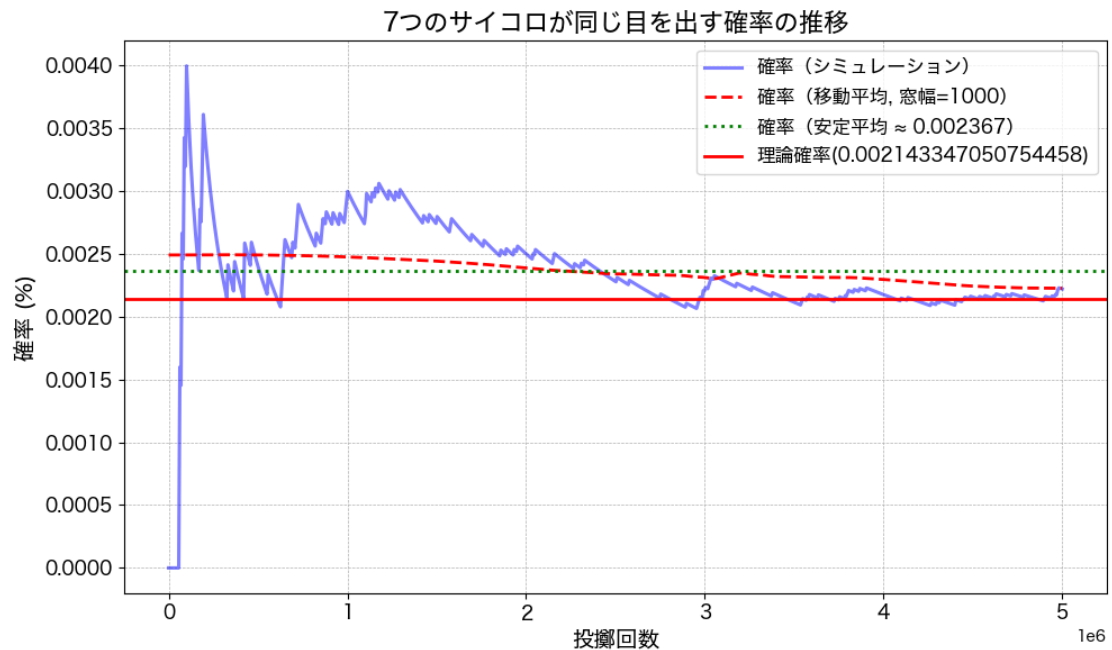


図 8: 投擲回数に対する 7つのサイコロが同じ目を出す確率

図 8 に示すように、投擲回数が少ない段階では出現率に大きな変動と揺らぎが見られる。しかし、繰り返し回数が増加するにつれて、グラフは安定し、変化率も著しく低下する。

この減速傾向にもかかわらず、窓幅 1000 データポイントの単純移動平均により強調された全体的な傾向は、理論的な期待確率へと着実に収束している。

1 回の投擲におけるサイコロの個数を変更した際の挙動は、表 8 に示されている。

表 8: サイコロの個数ごとの一致確率とその偏差 (5 百万回)

サイコロの個数	理論確率 (%)	実測値 (%)	偏差 (%)
2	16.666666667	16.667500000	+0.00500
4	0.462962963	0.462260000	-0.15184
6	0.012860082	0.012360000	-3.88864
8	0.000357225	0.000580000	+62.36288
10	0.000009923	0.000500000	+4938.84800

表 8 からわかるように、サイコロの個数が 2 個、4 個、6 個である場合、500 万回のシミュレーションによって得られた出現率は、計算された理論確率または期待値

からの偏差が5%未満に収まっている。特に、2個および4個のサイコロを用いたシミュレーションでは、最終的な偏差は0.5%未満にまで抑えられている。

一方で、サイコロの個数が8個および10個となるシミュレーションでは、理論値からの偏差が著しく大きく、最終的な偏差はそれぞれ約62.36%、および4938.85%に達している。

4.2 課題2: 半径7の円の面積

節4.1の実験と同様に、本シミュレーションにおいても、ランダムな点を繰り返し生成し、そのうち何点が円の内部に含まれるかをカウントすることで確率を算出している。各繰り返し時点において、円内に収まった点の総数をその時点までの繰り返し回数で割ることで、出現比率が計算される。

異なる繰り返し回数における、円内に収まる点の比率または割合のサンプリング結果は、表9に示されている。

表9: 繰り返し回数ごとの内包率と理論値との偏差

試行回数	実測内包率 (%)	理論内包率 (%)	偏差 (%)
500 回	80.800000	78.539816	+2.8778
700 回	80.142857	78.539816	+2.0411
1000 回	80.600000	78.539816	+2.6231
2000 回	78.650000	78.539816	+0.1403
3000 回	79.500000	78.539816	+1.2225
4000 回	78.850000	78.539816	+0.3949

表9のシミュレーション結果を見ると、比較的少ない繰り返し回数である500回においても、偏差率は約2.88%に抑えられていることがわかる。さらに繰り返し回数を増加させることで、偏差率は徐々に低下し、4000回時点ではわずか0.39%にまで縮小した。

このシミュレーション結果をグラフ化したものは、図9に示されている。

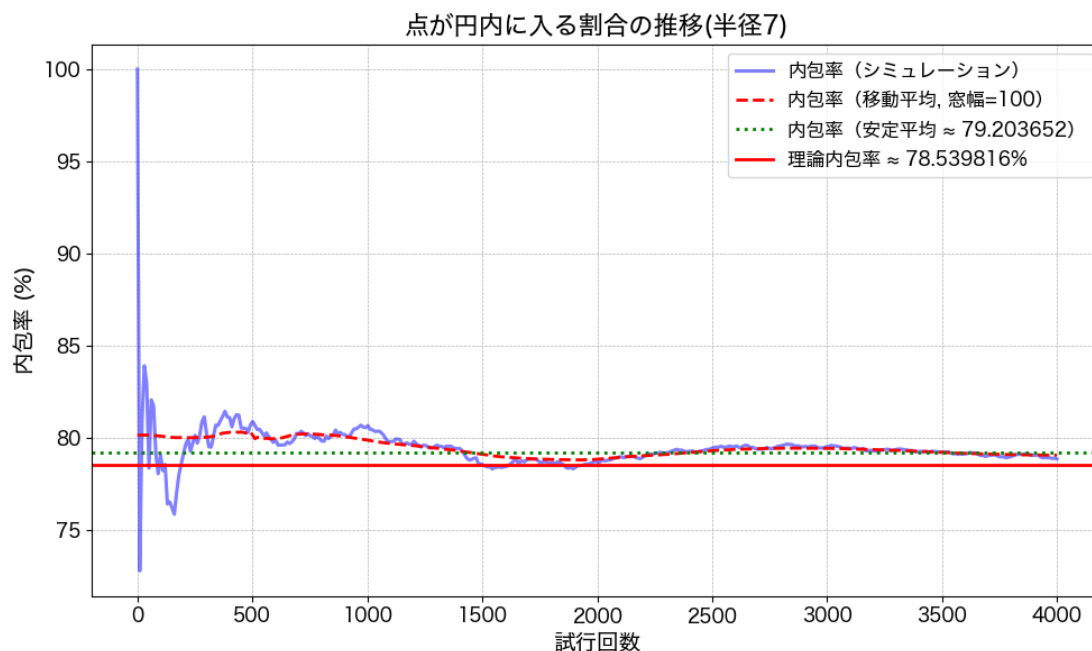


図 9: ランダム点が円内に属する割合 (半径 7 の円の面積)

図 9 に示すように、理論値からの偏差率は 5% 未満と比較的小さいものの、繰り返し回数が 1000 回程度の低い段階では、円内に点がプロットされる割合は依然として不安定である。しかし、繰り返し回数が 2000 回に近づくと、円内に収まる点の割合は大きく低下し、計算された理論値との偏差はわずか 1.22% にまで縮小した。

図 9 の 2000 回付近に着目すると、比率の推移はなお不安定であり、2500 回付近では再び理論値からの乖離が見られる。この段階以降、グラフの変動は徐々に安定しはじめるが、移動平均 (Simple Moving Average, SMA) を窓幅 100 で適用した線を通じて、理論値へと緩やかに収束し続けている様子が確認できる。

図 10 を通じて、2 次元空間内にプロットされた点の分布を観察できる。このグラフから、生成されたランダムな点はあらかじめ設定された範囲 -7 から 7 にかけて、均等かつ広く分布していることが確認できる。

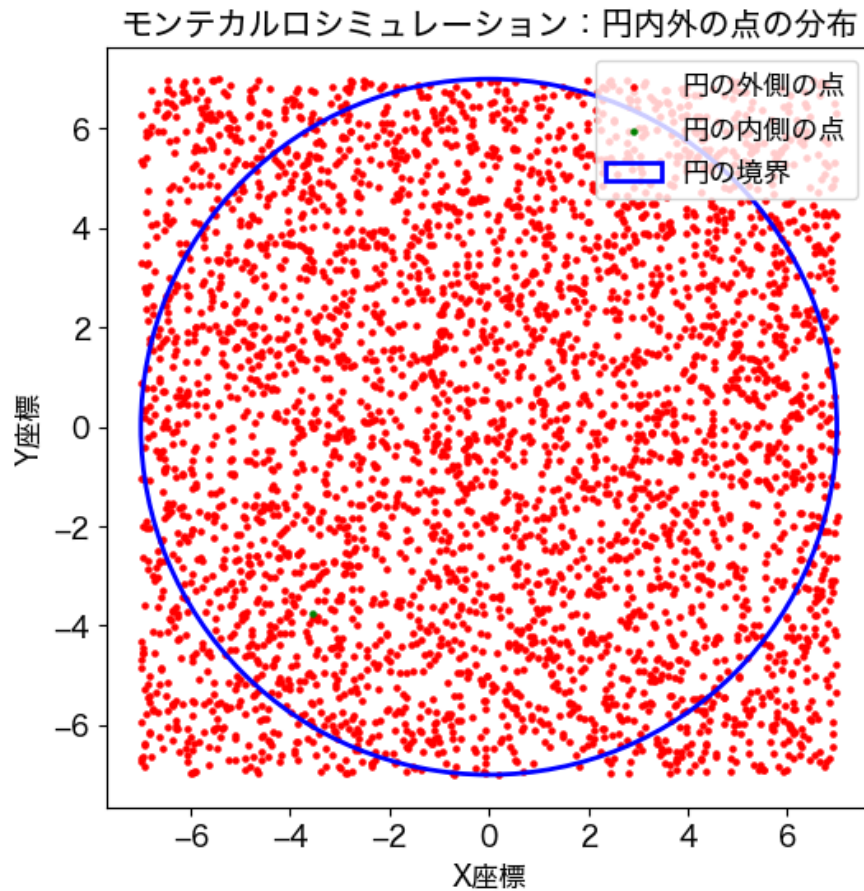


図 10: 2次元空間内にランダムにプロットされた点（半径7の円内）

参考文献

- [1] Wolfgang Hörmann and Gerhard Derflinger. A portable uniform random number generator well suited for the rejection method. WorkingPaper 3, Institut für Statistik und Mathematik, Abt. f. Angewandte Statistik u. Datenverarbeitung, WU Vienna University of Economics and Business, 1992.
- [2] IBM. What is monte carlo simulation?, 2021. Accessed: 2025-07-20.
- [3] Daniel Lemire. Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 29(1):1–12, 2019.
- [4] Marcin Majka. Moving averages in time series analysis. 09 2024.
- [5] Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, January 2019.

- [6] OpenBSD Project and BSD Manual Pages. *arc4random(3): High-quality CSPRNG functions*, 2025. Manual page, OpenBSD (and FreeBSD/macOS).
- [7] Giovanni Ossola and Alan D. Sokal. Systematic errors due to linear congruential random-number generators with the swendsen-wang algorithm: A warning. *Physical Review E*, 70(2), August 2004.
- [8] The OpenBSD Project. *arc4random* — cryptographic random number generator. <https://man.openbsd.org/arc4random>, 2024. Accessed: 2025-07-20.